BUILD CALCULATOR

Complete Python Source Code

```
import tkinter as tk
import math
import ast
import operator
from datetime import date
from tkinter import ttk


# ============================================================
# PROFESSIONAL SCIENTIFIC CALCULATOR
# ============================================================

root = tk.Tk()
root.title("Professional Calculator")
root.geometry("750x620")
root.resizable(False, False)
root.configure(bg="#13233C")


# ============================================================
# COLORS
# ============================================================

BG = "#13233C"
CARD = "#203B60"
BUTTON_BG = "#29496F"
ORANGE = "#F59E32"
WHITE = "#FFFFFF"
LIGHT = "#F2F5F9"
DARK_TEXT = "#172B4D"
GREEN = "#27AE60"
RED = "#E74C3C"


# ============================================================
# VARIABLES
# ============================================================

expression = tk.StringVar()
history_visible = False
history_data = []


# ============================================================
# MAIN FRAME
# ============================================================

main_frame = tk.Frame(root, bg=BG)
main_frame.pack(fill="both", expand=True, padx=12, pady=10)


# ============================================================
# DISPLAY
# ============================================================

display_frame = tk.Frame(
    main_frame,
    bg=CARD,
    padx=10,
    pady=10
)
display_frame.pack(fill="x", pady=5)

display = tk.Entry(
    display_frame,
    textvariable=expression,
    font=("Arial", 23, "bold"),
    bg=LIGHT,
    fg=DARK_TEXT,
    justify="right",
    bd=4,
    relief="solid"
)
display.pack(fill="x", ipady=8)


# ============================================================
# HISTORY
# ============================================================

history_frame = tk.Frame(display_frame, bg=CARD)

history_title = tk.Label(
    history_frame,
```

```
    text="CALCULATION HISTORY",
    font=("Arial", 12, "bold"),
    bg=CARD,
    fg=ORANGE
)
history_title.pack(anchor="w", padx=10, pady=5)

history_box = tk.Text(
    history_frame,
    height=5,
    font=("Arial", 10),
    bg=LIGHT,
    fg=DARK_TEXT,
    bd=0
)
history_box.pack(fill="x", padx=10, pady=5)


def add_history(text):
    history_data.append(text)

    history_box.config(state="normal")
    history_box.insert("end", text + "\n")
    history_box.see("end")
    history_box.config(state="disabled")


def toggle_history():
    global history_visible

    if history_visible:
        history_frame.pack_forget()
        history_visible = False
    else:
        history_frame.pack(fill="x", pady=5)
        history_visible = True


# ============================================================
# POPUP CENTER FUNCTION
# ============================================================

def center_popup(popup, width, height):

    popup.update_idletasks()

    x = root.winfo_x() + (root.winfo_width() - width) // 2
    y = root.winfo_y() + (root.winfo_height() - height) // 2

    popup.geometry(f"{width}x{height}+{x}+{y}")


# ============================================================
# PREMIUM MESSAGE POPUP
# ============================================================

def premium_popup(title_text, message, popup_type="info"):

    popup = tk.Toplevel(root)
    popup.title(title_text)
    popup.geometry("430x270")
    popup.configure(bg=BG)
    popup.resizable(False, False)
    popup.transient(root)
    popup.grab_set()

    center_popup(popup, 430, 270)

    if popup_type == "error":
        icon = "✕"
        icon_color = RED
        heading = "Something Went Wrong"
    else:
        icon = "✓"
        icon_color = GREEN
        heading = "Operation Completed"

    card = tk.Frame(
        popup,
        bg=CARD,
        padx=20,
        pady=15
    )
    card.pack(fill="both", expand=True, padx=12, pady=12)

    tk.Label(
        card,
        text=icon,
        font=("Arial", 25, "bold"),
        bg=icon_color,
        fg=WHITE,
        width=3,
```

```
        height=1
    ).pack(pady=(5, 10))

    tk.Label(
        card,
        text=heading,
        font=("Arial", 16, "bold"),
        bg=CARD,
        fg=ORANGE
    ).pack(pady=5)

    tk.Label(
        card,
        text=message,
        font=("Arial", 12, "bold"),
        bg=CARD,
        fg=WHITE,
        wraplength=360,
        justify="center"
    ).pack(pady=12)

    tk.Button(
        card,
        text="CONTINUE",
        font=("Arial", 12, "bold"),
        bg=ORANGE,
        fg=WHITE,
        activebackground="#E58A19",
        activeforeground=WHITE,
        bd=0,
        cursor="hand2",
        width=18,
        pady=9,
        command=popup.destroy
    ).pack(pady=5)

    popup.bind("&lt;Return&gt;", lambda event: popup.destroy())
    popup.bind("&lt;Escape&gt;", lambda event: popup.destroy())


def show_info(title_text, message):
    premium_popup(title_text, message, "info")


def show_error(title_text, message):
    premium_popup(title_text, message, "error")


# ============================================================
# PREMIUM INPUT POPUP
# ============================================================

def ask_float(title_text, prompt, minimum=None):

    popup = tk.Toplevel(root)
    popup.title(title_text)
    popup.geometry("430x280")
    popup.configure(bg=BG)
    popup.resizable(False, False)
    popup.transient(root)
    popup.grab_set()

    center_popup(popup, 430, 280)

    card = tk.Frame(
        popup,
        bg=CARD,
        padx=25,
        pady=20
    )
    card.pack(fill="both", expand=True, padx=12, pady=12)

    tk.Label(
        card,
        text=title_text.upper(),
        font=("Arial", 17, "bold"),
        bg=CARD,
        fg=ORANGE
    ).pack(pady=(5, 12))

    tk.Label(
        card,
        text=prompt,
        font=("Arial", 12, "bold"),
        bg=CARD,
        fg=WHITE
    ).pack(pady=5)

    entry = tk.Entry(
        card,
        font=("Arial", 17, "bold"),
        justify="center",
```

```
        bg=LIGHT,
        fg=DARK_TEXT,
        insertbackground=DARK_TEXT,
        relief="flat",
        bd=0
    )
    entry.pack(fill="x", ipady=10, pady=10)
    entry.focus_set()

    result = {"value": None}

    def submit():

        try:
            value = float(entry.get())

            if minimum is not None and value &lt; minimum:
                show_error(
                    "Invalid Input",
                    f"Please enter a value of at least {minimum}."
                )
                return

            result["value"] = value
            popup.destroy()

        except ValueError:
            show_error(
                "Invalid Input",
                "Please enter a valid number."
            )

    tk.Button(
        card,
        text="SUBMIT",
        font=("Arial", 12, "bold"),
        bg=ORANGE,
        fg=WHITE,
        activebackground="#E58A19",
        bd=0,
        cursor="hand2",
        width=18,
        pady=9,
        command=submit
    ).pack(pady=8)

    popup.bind("&lt;Return&gt;", lambda event: submit())
    popup.bind("&lt;Escape&gt;", lambda event: popup.destroy())

    popup.wait_window()

    return result["value"]


# ============================================================
# BASIC CALCULATOR FUNCTIONS
# ============================================================

def press(value):
    expression.set(expression.get() + str(value))


def clear():
    expression.set("")

    # History tab open hone par hi AC history delete karega.
    # Normal calculation ke dauran AC dabane par history safe rahegi.
    if history_visible:
        history_data.clear()
        history_box.config(state="normal")
        history_box.delete("1.0", "end")
        history_box.config(state="disabled")


def backspace():
    expression.set(expression.get()[:-1])


def toggle_sign():

    value = expression.get()

    if not value:
        return

    try:
        number = float(value)
        expression.set(str(-number))

    except ValueError:
        expression.set("-(" + value + ")")
# ============================================================
```

```
# SAFE CALCULATION
# ============================================================

allowed_operators = {
    ast.Add: operator.add,
    ast.Sub: operator.sub,
    ast.Mult: operator.mul,
    ast.Div: operator.truediv,
    ast.Pow: operator.pow,
    ast.Mod: operator.mod,
    ast.USub: operator.neg,
    ast.UAdd: operator.pos
}


def safe_eval(node):

    if isinstance(node, ast.Constant):

        if isinstance(node.value, (int, float)):
            return node.value

        raise ValueError("Invalid value")

    if isinstance(node, ast.BinOp):

        operation = allowed_operators.get(type(node.op))

        if operation is None:
            raise ValueError("Invalid operation")

        return operation(
            safe_eval(node.left),
            safe_eval(node.right)
        )

    if isinstance(node, ast.UnaryOp):

        operation = allowed_operators.get(type(node.op))

        if operation is None:
            raise ValueError("Invalid operation")

        return operation(safe_eval(node.operand))

    raise ValueError("Invalid expression")


def calculate():

    try:
        value = expression.get()

        if not value:
            return

        value = value.replace("×", "*")
        value = value.replace("÷", "/")
        value = value.replace("π", str(math.pi))

        tree = ast.parse(value, mode="eval")
        result = safe_eval(tree.body)

        expression.set(str(result))
        add_history(f"{value} = {result}")

    except ZeroDivisionError:
        show_error("Math Error", "Cannot divide by zero.")

    except Exception:
        show_error("Error", "Invalid mathematical expression.")


# ============================================================
# SCIENTIFIC FUNCTIONS
# ============================================================

def scientific_function(function_name):

    try:
        value = float(expression.get())

        if function_name == "sin":
            result = math.sin(math.radians(value))

        elif function_name == "cos":
            result = math.cos(math.radians(value))

        elif function_name == "tan":
            result = math.tan(math.radians(value))

        elif function_name == "log":
```

```
            if value &lt;= 0:
                show_error(
                    "Math Error",
                    "Log requires a positive number."
                )
                return

            result = math.log10(value)

        elif function_name == "sqrt":

            if value &lt; 0:
                show_error(
                    "Math Error",
                    "Square root of negative number is invalid."
                )
                return

            result = math.sqrt(value)

        expression.set(str(result))
        add_history(f"{function_name}({value}) = {result}")

    except ValueError:
        show_error(
            "Input Error",
            "Please enter a valid number."
        )


def percentage():

    try:
        value = float(expression.get())
        result = value / 100

        expression.set(str(result))
        add_history(f"{value}% = {result}")

    except ValueError:
        show_error(
            "Input Error",
            "Please enter a valid number."
        )


def insert_pi():
    press(math.pi)


# ============================================================
# GST CALCULATOR
# ============================================================

def gst_calculation():

    amount = ask_float(
        "GST Calculator",
        "Enter Amount (■):",
        0
    )

    if amount is None:
        return

    rate = ask_float(
        "GST Calculator",
        "Enter GST Rate (%):",
        0
    )

    if rate is None:
        return

    gst_amount = amount * rate / 100
    total = amount + gst_amount

    message = (
        f"Original Amount: ■{amount:,.2f}\n\n"
        f"GST Rate: {rate}%\n\n"
        f"GST Amount: ■{gst_amount:,.2f}\n\n"
        f"Final Amount: ■{total:,.2f}"
    )

    add_history(
        f"GST on ■{amount} at {rate}% = ■{total:.2f}"
    )

    show_info("GST Calculation Result", message)


# ============================================================
```

```
# LOAN EMI CALCULATOR
# ============================================================

def loan_emi():

    principal = ask_float(
        "Loan EMI Calculator",
        "Enter Loan Amount (■):",
        0
    )

    if principal is None:
        return

    annual_rate = ask_float(
        "Loan EMI Calculator",
        "Enter Annual Interest Rate (%):",
        0
    )

    if annual_rate is None:
        return

    years = ask_float(
        "Loan EMI Calculator",
        "Enter Loan Duration (Years):",
        0
    )

    if years is None or years &lt;= 0:
        show_error(
            "Invalid Input",
            "Loan duration must be greater than zero."
        )
        return

    months = years * 12
    monthly_rate = annual_rate / 12 / 100

    if monthly_rate == 0:
        emi = principal / months
    else:
        emi = (
            principal * monthly_rate *
            (1 + monthly_rate) ** months
        ) / (
            (1 + monthly_rate) ** months - 1
        )

    total_payment = emi * months
    total_interest = total_payment - principal

    message = (
        f"Loan Amount: ■{principal:,.2f}\n\n"
        f"Monthly EMI: ■{emi:,.2f}\n\n"
        f"Total Interest: ■{total_interest:,.2f}\n\n"
        f"Total Payment: ■{total_payment:,.2f}"
    )

    add_history(f"Loan EMI = ■{emi:.2f}")

    show_info("Loan EMI Result", message)


# ============================================================
# AGE CALCULATOR - PREMIUM POPUP
# ============================================================

def age_calculation():

    popup = tk.Toplevel(root)
    popup.title("Age Calculator")
    popup.geometry("460x430")
    popup.configure(bg=BG)
    popup.resizable(False, False)
    popup.transient(root)
    popup.grab_set()

    center_popup(popup, 460, 430)

    card = tk.Frame(
        popup,
        bg=CARD,
        padx=25,
        pady=20
    )
    card.pack(fill="both", expand=True, padx=12, pady=12)

    tk.Label(
        card,
        text="AGE CALCULATOR",
        font=("Arial", 20, "bold"),
```

```
    bg=CARD,
    fg=ORANGE
).pack(pady=(5, 5))

tk.Label(
    card,
    text="Enter your date of birth to calculate your age.",
    font=("Arial", 10, "bold"),
    bg=CARD,
    fg=WHITE,
    wraplength=380
).pack(pady=(0, 15))

# DATE INPUTS

input_frame = tk.Frame(card, bg=CARD)
input_frame.pack(fill="x", pady=10)

input_frame.grid_columnconfigure(0, weight=1)
input_frame.grid_columnconfigure(1, weight=1)
input_frame.grid_columnconfigure(2, weight=1)

tk.Label(
    input_frame,
    text="DAY",
    font=("Arial", 11, "bold"),
    bg=CARD,
    fg=WHITE
).grid(row=0, column=0, pady=8)

tk.Label(
    input_frame,
    text="MONTH",
    font=("Arial", 11, "bold"),
    bg=CARD,
    fg=WHITE
).grid(row=0, column=1, pady=8)

tk.Label(
    input_frame,
    text="YEAR",
    font=("Arial", 11, "bold"),
    bg=CARD,
    fg=WHITE
).grid(row=0, column=2, pady=8)

day_entry = tk.Entry(
    input_frame,
    font=("Arial", 16, "bold"),
    justify="center",
    bg=LIGHT,
    fg=DARK_TEXT,
    relief="flat",
    width=7
)
day_entry.grid(row=1, column=0, padx=5, ipady=10)

month_entry = tk.Entry(
    input_frame,
    font=("Arial", 16, "bold"),
    justify="center",
    bg=LIGHT,
    fg=DARK_TEXT,
    relief="flat",
    width=7
)
month_entry.grid(row=1, column=1, padx=5, ipady=10)

year_entry = tk.Entry(
    input_frame,
    font=("Arial", 16, "bold"),
    justify="center",
    bg=LIGHT,
    fg=DARK_TEXT,
    relief="flat",
    width=7
)
year_entry.grid(row=1, column=2, padx=5, ipady=10)

tk.Label(
    card,
    text="Example: Day = 15 | Month = 08 | Year = 2000",
    font=("Arial", 9),
    bg=CARD,
    fg="#D6E2F0"
).pack(pady=8)

def calculate_age():

    try:
        day = int(day_entry.get())
        month = int(month_entry.get())
```

```
            year = int(year_entry.get())

            dob = date(year, month, day)
            today = date.today()

            if dob &gt; today:
                show_error(
                    "Invalid Date",
                    "Date of birth cannot be in the future."
                )
                return

            years = today.year - dob.year
            months = today.month - dob.month
            days = today.day - dob.day

            if days &lt; 0:

                months -= 1

                if today.month == 1:
                    previous_month = 12
                    previous_year = today.year - 1
                else:
                    previous_month = today.month - 1
                    previous_year = today.year

                days += (
                    date(today.year, today.month, 1) -
                    date(previous_year, previous_month, 1)
                ).days

            if months &lt; 0:
                years -= 1
                months += 12

            message = (
                f"Your Current Age:\n\n"
                f"{years} Years\n"
                f"{months} Months\n"
                f"{days} Days"
            )

            add_history(
                f"Age = {years} Years, {months} Months, {days} Days"
            )

            popup.destroy()
            show_info("Your Age Result", message)

        except ValueError:
            show_error(
                "Invalid Date",
                "Please enter a valid date.\n"
                "Example: 15 / 08 / 2000"
            )

    tk.Button(
        card,
        text="CALCULATE AGE",
        font=("Arial", 13, "bold"),
        bg=ORANGE,
        fg=WHITE,
        activebackground="#E58A19",
        activeforeground=WHITE,
        bd=0,
        cursor="hand2",
        width=22,
        pady=11,
        command=calculate_age
    ).pack(pady=15)

    tk.Button(
        card,
        text="CLOSE",
        font=("Arial", 10, "bold"),
        bg=BUTTON_BG,
        fg=WHITE,
        bd=0,
        cursor="hand2",
        width=12,
        pady=7,
        command=popup.destroy
    ).pack()


# ============================================================
# BMI CALCULATOR
# ============================================================

def bmi_calculation():
    weight = ask_float(
```

```
        "BMI Calculator",
        "Enter Weight (kg):",
        0
    )

    if weight is None:
        return

    if weight &lt;= 0:
        show_error(
            "Invalid Input",
            "Weight must be greater than zero."
        )
        return

    height = ask_float(
        "BMI Calculator",
        "Enter Height (cm):",
        0
    )

    if height is None:
        return

    if height &lt;= 0:
        show_error(
            "Invalid Input",
            "Height must be greater than zero."
        )
        return

    bmi = weight / ((height / 100) ** 2)

    if bmi &lt; 18.5:
        category = "Underweight"
    elif bmi &lt; 25:
        category = "Normal Weight"
    elif bmi &lt; 30:
        category = "Overweight"
    else:
        category = "Obese"

    message = (
        f"Weight: {weight} kg\n\n"
        f"Height: {height} cm\n\n"
        f"BMI Score: {bmi:.2f}\n\n"
        f"Category: {category}"
    )

    add_history(f"BMI = {bmi:.2f} ({category})")

    show_info("BMI Calculation Result", message)


# ============================================================
# UNIT CONVERTER DATA
# ============================================================

conversion_data = {

    "Length": {
        "Meter": 1,
        "Kilometer": 1000,
        "Centimeter": 0.01,
        "Millimeter": 0.001,
        "Mile": 1609.344,
        "Foot": 0.3048,
        "Inch": 0.0254
    },

    "Weight": {
        "Kilogram": 1,
        "Gram": 0.001,
        "Milligram": 0.000001,
        "Pound": 0.45359237,
        "Ounce": 0.0283495
    },

    "Time": {
        "Second": 1,
        "Minute": 60,
        "Hour": 3600,
        "Day": 86400,
        "Week": 604800
    },

    "Speed": {
        "Meter/Second": 1,
        "Kilometer/Hour": 0.2777778,
        "Mile/Hour": 0.44704
    },
    "Volume": {
```

```
        "Liter": 1,
        "Milliliter": 0.001,
        "Cubic Meter": 1000,
        "Gallon (US)": 3.78541
    }
}


# ============================================================
# TEMPERATURE CONVERSION
# ============================================================

def convert_temperature(value, from_unit, to_unit):

    if from_unit == "Celsius":
        celsius = value

    elif from_unit == "Fahrenheit":
        celsius = (value - 32) * 5 / 9

    else:
        celsius = value - 273.15

    if to_unit == "Celsius":
        return celsius

    elif to_unit == "Fahrenheit":
        return (celsius * 9 / 5) + 32

    else:
        return celsius + 273.15


# ============================================================
# CONVERTER - PREMIUM POPUP
# ============================================================

def converter_window():

    converter = tk.Toplevel(root)
    converter.title("Unit Converter")
    converter.geometry("520x650")
    converter.configure(bg=BG)
    converter.resizable(False, False)
    converter.transient(root)
    converter.grab_set()

    center_popup(converter, 520, 650)

    card = tk.Frame(
        converter,
        bg=CARD,
        padx=25,
        pady=18
    )
    card.pack(fill="both", expand=True, padx=12, pady=12)

    tk.Label(
        card,
        text="UNIT CONVERTER",
        font=("Arial", 20, "bold"),
        bg=CARD,
        fg=ORANGE
    ).pack(pady=(5, 5))

    tk.Label(
        card,
        text="Convert units quickly and easily.",
        font=("Arial", 10, "bold"),
        bg=CARD,
        fg=WHITE
    ).pack(pady=(0, 15))

    # CATEGORY

    tk.Label(
        card,
        text="1. SELECT CONVERSION CATEGORY",
        font=("Arial", 11, "bold"),
        bg=CARD,
        fg=WHITE
    ).pack(anchor="w", pady=(8, 5))

    category_var = tk.StringVar(value="Length")

    category_box = ttk.Combobox(
        card,
        textvariable=category_var,
        values=[
            "Length",
            "Weight",
            "Time",
```

```
        "Speed",
        "Volume",
        "Temperature",
        "Currency"
    ],
    state="readonly",
    font=("Arial", 12)
)
category_box.pack(fill="x", ipady=5)

# AMOUNT

tk.Label(
    card,
    text="2. ENTER AMOUNT",
    font=("Arial", 11, "bold"),
    bg=CARD,
    fg=WHITE
).pack(anchor="w", pady=(15, 5))

amount_entry = tk.Entry(
    card,
    font=("Arial", 16, "bold"),
    justify="center",
    bg=LIGHT,
    fg=DARK_TEXT,
    relief="flat"
)
amount_entry.pack(fill="x", ipady=8)

# FROM UNIT

tk.Label(
    card,
    text="3. CONVERT FROM",
    font=("Arial", 11, "bold"),
    bg=CARD,
    fg=WHITE
).pack(anchor="w", pady=(15, 5))

from_var = tk.StringVar()

from_box = ttk.Combobox(
    card,
    textvariable=from_var,
    state="readonly",
    font=("Arial", 12)
)
from_box.pack(fill="x", ipady=5)

# TO UNIT

tk.Label(
    card,
    text="4. CONVERT TO",
    font=("Arial", 11, "bold"),
    bg=CARD,
    fg=WHITE
).pack(anchor="w", pady=(15, 5))

to_var = tk.StringVar()

to_box = ttk.Combobox(
    card,
    textvariable=to_var,
    state="readonly",
    font=("Arial", 12)
)
to_box.pack(fill="x", ipady=5)

# CURRENCY RATE

currency_frame = tk.Frame(card, bg=CARD)

tk.Label(
    currency_frame,
    text="Exchange Rate (1 From Unit = ? To Unit)",
    font=("Arial", 10, "bold"),
    bg=CARD,
    fg=ORANGE
).pack(anchor="w", pady=5)

currency_rate_entry = tk.Entry(
    currency_frame,
    font=("Arial", 13, "bold"),
    justify="center",
    bg=LIGHT,
    fg=DARK_TEXT
)
currency_rate_entry.pack(fill="x", ipady=7)

# RESULT
```

```
result_label = tk.Label(
    card,
    text="Your conversion result will appear here.",
    font=("Arial", 12, "bold"),
    bg=BG,
    fg=ORANGE,
    wraplength=420,
    padx=10,
    pady=12
)
result_label.pack(fill="x", pady=15)

# UPDATE UNITS

def update_units(event=None):

    category = category_var.get()

    if category == "Temperature":

        units = [
            "Celsius",
            "Fahrenheit",
            "Kelvin"
        ]

    elif category == "Currency":

        units = [
            "INR",
            "USD",
            "EUR",
            "GBP",
            "AED",
            "JPY"
        ]

    else:
        units = list(
            conversion_data[category].keys()
        )

    from_box["values"] = units
    to_box["values"] = units

    from_var.set(units[0])
    to_var.set(units[1])

    if category == "Currency":

        currency_frame.pack(
            fill="x",
            pady=8
        )

        currency_rate_entry.delete(0, "end")
        currency_rate_entry.insert(0, "1")

    else:
        currency_frame.pack_forget()

# CONVERSION FUNCTION

def perform_conversion():

    try:
        amount = float(amount_entry.get())

        category = category_var.get()
        from_unit = from_var.get()
        to_unit = to_var.get()

        if category == "Temperature":

            result = convert_temperature(
                amount,
                from_unit,
                to_unit
            )

        elif category == "Currency":

            rate = float(currency_rate_entry.get())

            if rate &lt;= 0:
                show_error(
                    "Invalid Exchange Rate",
                    "Please enter an exchange rate greater than zero."
                )
                return

            result = amount * rate
```

```
            else:

                units = conversion_data[category]

                result = (
                    amount *
                    units[from_unit] /
                    units[to_unit]
                )

            result_label.config(
                text=(
                    f"CONVERSION RESULT\n\n"
                    f"{amount:g} {from_unit}\n"
                    f"=\n"
                    f"{result:.6g} {to_unit}"
                )
            )

            add_history(
                f"Convert: {amount:g} {from_unit} = "
                f"{result:.6g} {to_unit}"
            )

        except ValueError:

            show_error(
                "Invalid Input",
                "Please enter a valid amount and exchange rate."
            )

    category_box.bind(
        "&lt;&lt;ComboboxSelected&gt;&gt;",
        update_units
    )

    update_units()

    # BUTTONS

    tk.Button(
        card,
        text="CONVERT NOW",
        font=("Arial", 13, "bold"),
        bg=ORANGE,
        fg=WHITE,
        activebackground="#E58A19",
        activeforeground=WHITE,
        bd=0,
        cursor="hand2",
        pady=11,
        command=perform_conversion
    ).pack(fill="x", pady=5)

    tk.Button(
        card,
        text="CLOSE",
        font=("Arial", 10, "bold"),
        bg=BUTTON_BG,
        fg=WHITE,
        bd=0,
        cursor="hand2",
        pady=8,
        command=converter.destroy
    ).pack(fill="x", pady=5)

    tk.Label(
        converter,
        text="Currency exchange rate is entered manually.",
        font=("Arial", 9),
        bg=BG,
        fg=WHITE
    ).pack(pady=3)


# ============================================================
# BUTTON CREATION FUNCTION
# ============================================================

def create_button(
    parent,
    text,
    row,
    column,
    command,
    bg=BUTTON_BG,
    fg=WHITE
):

    button = tk.Button(
        parent,
        text=text,
```

```
        font=("Arial", 12, "bold"),
        bg=bg,
        fg=fg,
        activebackground=ORANGE,
        activeforeground=WHITE,
        bd=0,
        relief="flat",
        cursor="hand2",
        command=command
    )

    button.grid(
        row=row,
        column=column,
        padx=4,
        pady=4,
        sticky="nsew",
        ipadx=3,
        ipady=9
    )

    return button


# ============================================================
# BUTTON FRAME
# ============================================================

button_frame = tk.Frame(
    main_frame,
    bg=BG
)
button_frame.pack(
    fill="both",
    expand=True,
    pady=5
)

# Equal width for all columns

for i in range(5):

    button_frame.grid_columnconfigure(
        i,
        weight=1,
        uniform="buttons"
    )

# Equal height for all rows

for i in range(7):

    button_frame.grid_rowconfigure(
        i,
        weight=1,
        uniform="buttons"
    )


# ============================================================
# BUTTON LAYOUT
# ============================================================

buttons = [

    # ROW 1
    ("LOAN EMI", 0, 0, loan_emi, ORANGE),
    ("GST CAL", 0, 1, gst_calculation, ORANGE),
    ("AGE CAL", 0, 2, age_calculation, ORANGE),
    ("BMI", 0, 3, bmi_calculation, ORANGE),
    ("CONVERT", 0, 4, converter_window, ORANGE),

    # ROW 2
    ("log", 1, 0, lambda: scientific_function("log"), BUTTON_BG),
    ("sin", 1, 1, lambda: scientific_function("sin"), BUTTON_BG),
    ("cos", 1, 2, lambda: scientific_function("cos"), BUTTON_BG),
    ("tan", 1, 3, lambda: scientific_function("tan"), BUTTON_BG),
    ("HISTORY", 1, 4, toggle_history, ORANGE),

    # ROW 3
    ("AC", 2, 0, clear, RED),
    ("(", 2, 1, lambda: press("("), BUTTON_BG),
    (")", 2, 2, lambda: press(")"), BUTTON_BG),
    ("%", 2, 3, percentage, BUTTON_BG),
    ("=", 2, 4, calculate, GREEN),

    # ROW 4
    ("7", 3, 0, lambda: press("7"), BUTTON_BG),
    ("8", 3, 1, lambda: press("8"), BUTTON_BG),
    ("9", 3, 2, lambda: press("9"), BUTTON_BG),
    ("÷", 3, 3, lambda: press("/"), BUTTON_BG),
    ("■", 3, 4, backspace, RED),
```

```
    # ROW 5
    ("4", 4, 0, lambda: press("4"), BUTTON_BG),
    ("5", 4, 1, lambda: press("5"), BUTTON_BG),
    ("6", 4, 2, lambda: press("6"), BUTTON_BG),
    ("×", 4, 3, lambda: press("*"), BUTTON_BG),
    ("√", 4, 4, lambda: scientific_function("sqrt"), ORANGE),

    # ROW 6
    ("1", 5, 0, lambda: press("1"), BUTTON_BG),
    ("2", 5, 1, lambda: press("2"), BUTTON_BG),
    ("3", 5, 2, lambda: press("3"), BUTTON_BG),
    ("+", 5, 3, lambda: press("+"), BUTTON_BG),
    ("π", 5, 4, insert_pi, ORANGE),

    # ROW 7
    ("00", 6, 0, lambda: press("00"), BUTTON_BG),
    ("0", 6, 1, lambda: press("0"), BUTTON_BG),
    (".", 6, 2, lambda: press("."), BUTTON_BG),
    ("−", 6, 3, lambda: press("-"), BUTTON_BG),
    ("±", 6, 4, toggle_sign, ORANGE)
]


for text, row, column, command, color in buttons:

    create_button(
        button_frame,
        text,
        row,
        column,
        command,
        bg=color
    )


# ============================================================
# KEYBOARD SUPPORT
# ============================================================

def keyboard_input(event):

    if event.keysym == "Return":
        calculate()
        return "break"

    elif event.keysym == "BackSpace":
        backspace()
        return "break"

    elif event.keysym == "Escape":
        clear()
        return "break"


root.bind("&lt;Return&gt;", keyboard_input)
root.bind("&lt;BackSpace&gt;", keyboard_input)
root.bind("&lt;Escape&gt;", keyboard_input)


# ============================================================
# FOOTER
# ============================================================

footer = tk.Label(
    main_frame,
    text="Build Calculator",
    font=("Arial", 8, "bold"),
    bg=BG,
    fg="#F2F4F6"
)
footer.pack(pady=2)


# ============================================================
# START APPLICATION
# ============================================================

root.mainloop()
```